

Expt. No. : 11

Date :

PIVOT TABLE USING TITANIC DATASET

AIM

To write a Python program to create and analyse a pivot table using the Pandas library on the Titanic dataset.

SOFTWARE REQUIRED

Python 3.14

THEORY

Pivot tables are a powerful tool in data analysis that allow you to summarize data in a flexible manner. In Pandas, pivot tables function similarly to those in spreadsheet programs like Microsoft Excel.

Syntax for Creating a Pivot Table

To create a pivot table in Pandas, you use the `pd.pivot_table()` function. Here is a basic example of the syntax:

```
table = pd.pivot_table(df,
                        values=['values_column'],
                        index=['row_column_1', 'row_column_2'],
                        columns=['column_column'],
                        aggfunc='aggregate_function')
```

- **df:** The DataFrame containing the data.
- **values:** The column(s) you want to aggregate, typically numerical data.
- **index:** The column(s) you want as rows in the pivot table (row-level grouping).
- **columns:** The column(s) you want to be displayed as columns (column-level grouping).
- **aggfunc:** The aggregation function to apply (such as mean, sum, count, etc.).

Elements of a Pivot Table

1. **Index:** This specifies how rows are grouped. It can be a single column or multiple columns from your DataFrame.
2. **Columns:** This groups the data into columns, which can also consist of multiple columns.
3. **Values:** These are the numerical values you want to aggregate, such as the sum or average of a dataset's numerical column.

Aggregation Function: This is the function used to summarize the data. Common functions include mean, sum, count, min, and max.

PROBLEM

1. Build a Python program using Pandas to create a Pivot Table that compares the average fare paid by survived and non-survived passengers across different passenger classes.
2. Build a Python program using Pandas to create a Pivot Table that calculates the total energy generated by solar panels based on panel type and time of day.

DATASET

```
data = {
    "Equipment_ID": ["E001", "E002", "E003", "E004", "E005", "E006", "E007", "E008",
                    "E009", "E010", "E011", "E012", "E013", "E014", "E015"],
    "Equipment_Type": ["Transformer", "Motor", "Transformer", "Motor", "Solar Panel",
                      "Transformer", "Motor", "Solar Panel", "Transformer", "Motor", "Solar Panel",
                      "Transformer", "Motor", "Solar Panel", "Transformer"],
    "Feeder": ["F1", "F1", "F2", "F2", "F3", "F1", "F2", "F3", "F2", "F1", "F3", "F1", "F2",
              "F3", "F2"],
    "Shift": ["Morning", "Morning", "Afternoon", "Afternoon", "Evening", "Morning",
             "Afternoon", "Evening", "Morning", "Afternoon", "Evening", "Afternoon", "Morning",
             "Afternoon", "Evening"],
    "Voltage_V": [230, 232, 235, 228, 225, 231, 229, 224, 236, 230, 226, 233, 227, 223,
                  234],
    "Current_A": [12.5, 15.2, 13.8, 16.1, 8.5, 12.9, 15.8, 8.2, 14.1, 16.4, 8.7, 13.2, 15.5, 8.9,
                  13.6],
    "Power_kW": [2.75, 3.52, 3.10, 3.67, 1.91, 2.98, 3.62, 1.84, 3.32, 3.74, 1.96, 3.08, 3.51,
                 1.88, 3.18],
    "Temperature_C": [32, 38, 41, 43, 35, 34, 40, 36, 42, 44, 37, 39, 41, 34, 43],
    "Efficiency_Percent": [94, 91, 95, 89, 87, 93, 90, 88, 96, 89, 86, 94, 92, 89, 95],
    "Power_Factor": [0.92, 0.88, 0.94, 0.86, 0.96, 0.91, 0.87, 0.95, 0.93, 0.85, 0.97, 0.90,
                     0.89, 0.94, 0.92],
    "Energy_Consumption_kWh": [22.0, 28.2, 24.8, 29.4, 15.3, 23.8, 28.9, 14.7, 26.6,
                               30.0, 15.7, 24.6, 28.1, 15.0, 25.4]
}
```

3. Pivot the *Occupation* column in OCCUPATIONS so that each *Name* is sorted alphabetically and displayed underneath its corresponding *Occupation*. The output should consist of four columns (*Doctor*, *Professor*, *Singer*, and *Actor*) in that specific order, with their respective names listed alphabetically under each column.

Note: Print NULL when there are no more names corresponding to an occupation.

ALGORITHM

STEP 1: Start the program.

STEP 2: Import pandas as pd

STEP 3: Get the titanic dataset

STEP 4: Print the first few rows of the dataset

STEP 5: Create a pivot table using the function `pd.pivot_table()`

STEP 6: Specify the data frame, index, columns, values and aggregation functions
as

arguments to the function

STEP 7: Print the pivot table

STEP 8: Additional analysis using pivot tables can be done with different
combinations

of arguments

STEP 9: Terminate the program.

PROGRAM CODING

```

import pandas as pd
n = int(input("Enter number of passengers: "))
data = []
for i in range(n):
    pclass = int(input("Enter passenger class: "))
    survived = int(input("Enter survived (0 or 1): "))
    fare = float(input("Enter fare: "))
    data.append([pclass, survived, fare])
df = pd.DataFrame(data, columns=["Pclass", "Survived", "Fare"])
table = pd.pivot_table(
    df,
    values="Fare",
    index="Pclass",
    columns="Survived",
    aggfunc="mean"
)
print("\nPivot Table:")
print(table)

```

OUTCOMES OF PROGRAM CODING

```

IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\Mugil\Desktop\Python\Experiment\Experiment11.py =====
Enter number of passengers: 4
Enter passenger class: 1
Enter survived (0 or 1): 1
Enter fare: 100
Enter passenger class: 1
Enter survived (0 or 1): 0
Enter fare: 80
Enter passenger class: 2
Enter survived (0 or 1): 1
Enter fare: 60
Enter passenger class: 2
Enter survived (0 or 1): 0
Enter fare: 40

Pivot Table:
Survived    0    1
Pclass
1          80.0 100.0
2          40.0  60.0

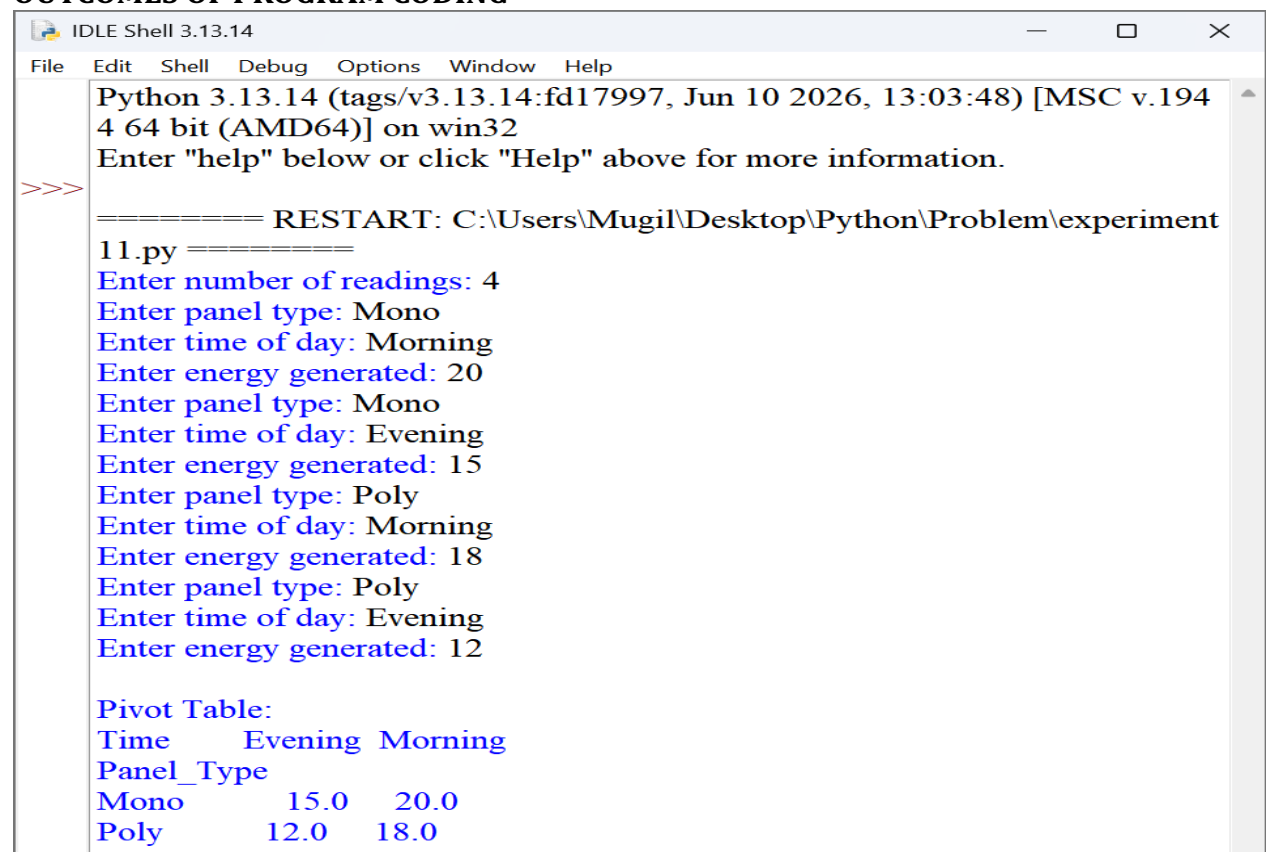
```

PROBLEM CODING

```

import pandas as pd
n = int(input("Enter number of readings: "))
data = []
for i in range(n):
    panel = input("Enter panel type: ")
    time = input("Enter time of day: ")
    energy = float(input("Enter energy generated: "))
    data.append([panel, time, energy])
df = pd.DataFrame(data, columns=["Panel_Type", "Time", "Energy"])
table = pd.pivot_table(
    df,
    values="Energy",
    index="Panel_Type",
    columns="Time",
    aggfunc="sum",
    fill_value=0
)
print("\nPivot Table:")
print(table)

```

OUTCOMES OF PROGRAM CODING


```

Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.194
4 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\Mugil\Desktop\Python\Problem\experiment
11.py =====
Enter number of readings: 4
Enter panel type: Mono
Enter time of day: Morning
Enter energy generated: 20
Enter panel type: Mono
Enter time of day: Evening
Enter energy generated: 15
Enter panel type: Poly
Enter time of day: Morning
Enter energy generated: 18
Enter panel type: Poly
Enter time of day: Evening
Enter energy generated: 12

Pivot Table:
Time      Evening  Morning
Panel_Type
Mono          15.0    20.0
Poly          12.0    18.0

```

PROBLEM CODING

```

n = int(input("Enter number of people: "))
doctor = []
professor = []
singer = []
actor = []
for i in range(n):
    name = input("Enter name: ")
    occupation = input("Enter occupation: ")
    if occupation == "Doctor":
        doctor.append(name)
    elif occupation == "Professor":
        professor.append(name)
    elif occupation == "Singer":
        singer.append(name)
    elif occupation == "Actor":
        actor.append(name)
doctor.sort()
professor.sort()
singer.sort()
actor.sort()
m = max(len(doctor), len(professor), len(singer), len(actor))
for i in range(m):
    d = doctor[i] if i < len(doctor) else "NULL"
    p = professor[i] if i < len(professor) else "NULL"
    s = singer[i] if i < len(singer) else "NULL"
    a = actor[i] if i < len(actor) else "NULL"
    print(d, p, s, a)

```

OUTCOMES OF PROGRAM CODING

```

IDLE Shell 3.13.14
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\Mugil\Desktop\Python\Problem1\11.py =====
Enter number of people: 4
Enter name: Arun
Enter occupation: Doctor
Enter name: Bala
Enter occupation: Singer
Enter name: Charan
Enter occupation: Actor
Enter name: Divya
Enter occupation: Doctor
Arun NULL Bala Charan
Divya NULL NULL NULL

```

VIVA QUESTION

1. What is a pivot table, and why is it used in data analysis?

A pivot table is a data processing tool that allows users to summarize and analyze large datasets in a structured format. It enables the aggregation of data based on different dimensions, such as rows and columns, making it easier to identify trends, patterns, and insights. Pivot tables are commonly used in data analysis for tasks like summarizing sales data, analyzing survey results, and performing statistical analysis.

2. What are the key components of a pivot table in Pandas?

The key components of a pivot table in Pandas are:

1. Index: The column(s) used for grouping data in the rows of the pivot table.
2. Columns: The column(s) used for grouping data in the columns of the pivot table.
3. Values: The numerical data that is aggregated in the pivot table, such as sums, averages, or counts.
4. Aggregation Function: The function used to summarize the data, such as mean, sum, count, min, or max.

3. How do you create a pivot table in Pandas?

To create a pivot table in Pandas, you can use the `pd.pivot_table()` function. The syntax is as

```
table = pd.pivot_table(df,
                        values='values_column',
                        index='row_column',
                        columns='column_column',
                        aggfunc='aggregate_function')
```

Where `df` is the DataFrame containing the data, and you specify the columns for values, index, and columns along with the desired aggregation function.

4. Can a pivot table have multiple indices or columns? If yes, how?

Yes, a pivot table can have multiple indices or columns. You can specify lists of columns for both the index and columns parameters in the `pd.pivot_table()` function. For example:

```
pivot = pd.pivot_table(data,
                        values='price',
                        index=['num-of-doors', 'body-style'],
                        columns=['fuel-type', 'fuel-system'],
                        aggfunc='mean')
```

This creates a multi-level pivot table that summarizes data across multiple dimensions.

5. How can you handle missing values in a pivot table?

You can handle missing values in a pivot table by using the `fill_value` parameter in the `pd.pivot_table()` function. This allows you to specify a value that will replace any missing entries in the pivot table. For example:

```
pivot = pd.pivot_table(data,
                        values='Operational Time',
```

```
index='Component Type',  
columns='Manufacturer',  
aggfunc='mean',  
fill_value=0)
```

In this case, any missing values in the pivot table will be replaced with 0.

6. What insights can you derive from pivot tables?

Pivot tables allow for quick insights into data trends, such as:

- Comparative Analysis: Understanding differences in metrics across different categories or groups.
- Summarization: Easily summarizing large datasets to find averages, totals, or counts.
- Identifying Patterns: Spotting trends or anomalies within the data.
- Decision-Making: Facilitating data-driven decision-making by presenting clear and concise information.

7. What are the limitations of pivot tables?

Some limitations of pivot tables include:

- Complexity: For very large datasets or complex calculations, pivot tables can become unwieldy.
- Static Nature: Pivot tables do not automatically update with new data unless they are refreshed.

Limited Functionality: Certain advanced analyses may require additional programming or tools beyond pivot tables.

RESULT:

Thus, the Python program successfully created and implemented a Pivot Table using the Titanic dataset. The data was grouped based on passenger class and gender, and the average fare or survival rate was calculated using Pandas. The program was executed successfully, and the output was verified.

INFERENCE:

The experiment demonstrates that Pivot Tables provide an efficient method for summarizing, grouping, and analysing large datasets. Using Pandas, meaningful relationships between variables such as passenger class, gender, fare, and survival rate can be identified, facilitating effective data analysis and interpretation.

Assessment	Max. Marks	Marks Obtained
Conduction of Practical	15	
Report	5	
Viva	5	
Total Marks	25	
Signature of Lab In-Charge		

